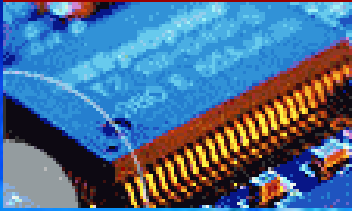
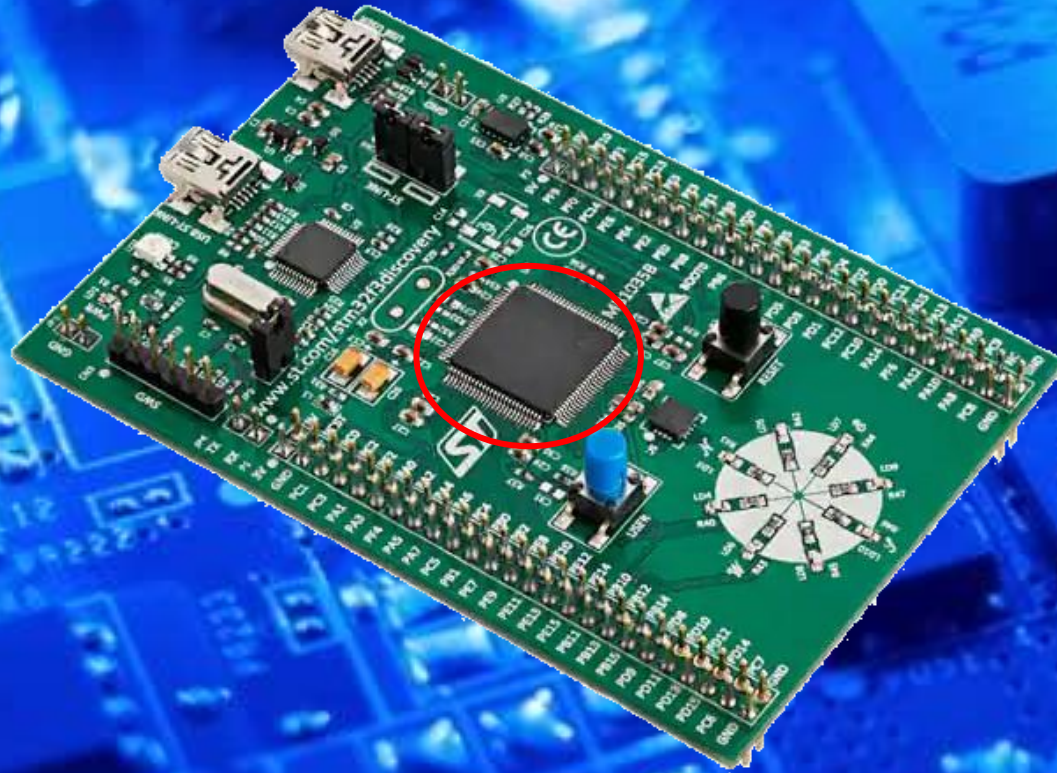


EEE3096S



**Embedded  
Systems II**



# Information Theory & Sampling

## Embedded Systems II

L07

Dr Simon Winberg



Electrical Engineering  
University of Cape Town

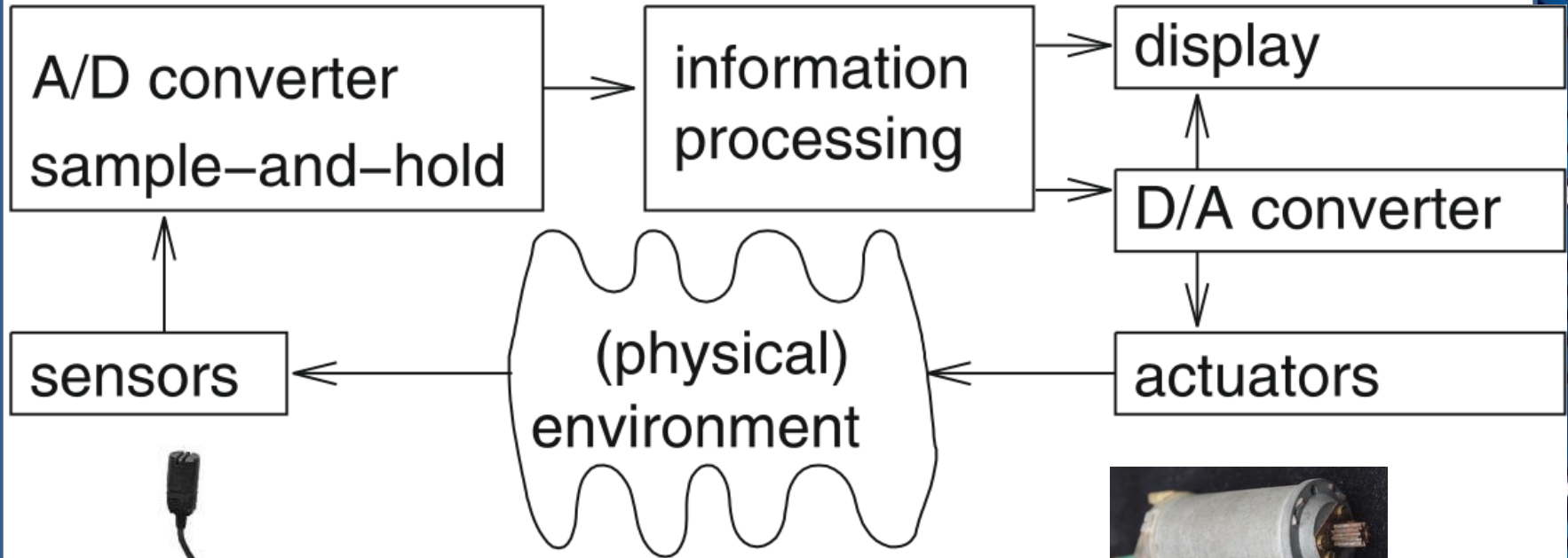
Planned as single period lecture

# Outline of Lecture

- Hardware in the loop (HITL)
- Scenario and HITL example
- Signals and Sampling signals
- Sample-and-hold circuits (also called 'S/H')
- Sampling Theory (brief recap)
- Shannon and Nyquist & relevance to ADCs

# Hardware-In-The-Loop (HITL)

Embedded system hardware is frequently used in a loop of sampling and actuating (i.e. ***“hardware in a loop”***):



☞ cyber-physical systems

Often HITL may have the ‘plant’ section partly emulated, feeding signals to the ES.

Explanation and further details at: [https://en.wikipedia.org/wiki/Hardware-in-the-loop\\_simulation](https://en.wikipedia.org/wiki/Hardware-in-the-loop_simulation)



# Many examples of such loops

- Heating
- Lights
- Engine control\*
- Power supply
- ...
- Robots



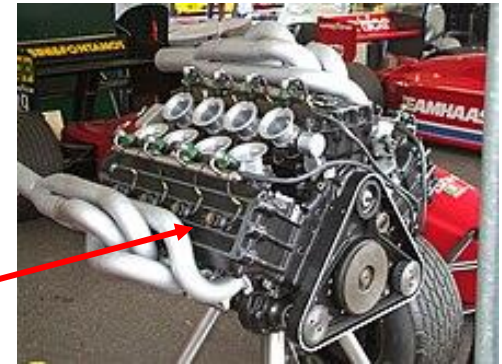
© P. Marwedel, 2011



\* let's take a look at an engine control scenario, to properly comprehend HITL ...

# Hardware In The Loop (HITL) Example: Race Car Engine Scenario

- Imagine you're in a team of engineers developing a new Engine Control Unit (ECU) for a high-performance car\*.
- ECU responsible for things like:
  - Controlling fuel injection,
  - ignition timing,
  - turbocharger boost pressure,
  - various other parameters to optimize engine performance, emissions, and fuel efficiency.



Engine view (W12 3.5 Formula 1)\*\*

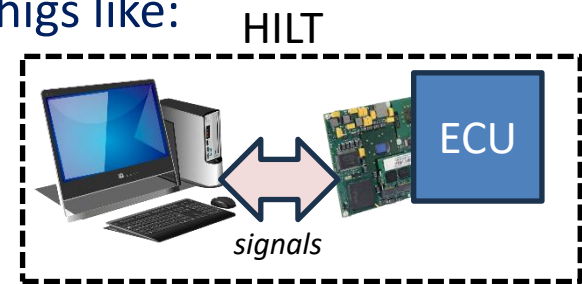
Develop an ES to  
control this thing  
super efficiently!

\* Sorry, can't help the acronyms tendency, just note CPU  $\neq$  ECU, ECU would likely, for a modern specialized engine, have many CPUs or rather microcontrollers. I don't mean to disappoint, but it's a petrol engine.

\*\* Image source: [https://commons.wikimedia.org/wiki/File:W12\\_Engine.jpg](https://commons.wikimedia.org/wiki/File:W12_Engine.jpg)

# HILT setup for Engine Control Unit (ECU)

1. Real ECU: actual engine control unit (hardware under test) connected to the HILT emulation setup (maybe connects to real thing, the car engine, but likely not feasible because the engine may be under construction at the same time).
2. Simulated Engine: computer runs sophisticated software that models the engine's behavior in real-time. Including things like:
  - Combustion processes
  - Thermodynamics
  - Fluid dynamics for intake & exhaust
  - Turbocharger dynamics
  - Mechanical components (crankshaft, pistons, valves)
3. Sensor Emulation: (or output to sensor emulation) HILT simulates the electrical signals of various engine sensors
4. Actuator Emulation: HILT emulates behavior of actuators that the ECU controls
5. Real-time Interaction: The ECU interacts with the simulated engine as if it was the real thing.  
(i.e. feedback loop as in sensor->process->actuator->sensor-> ...)





# Benefits of HITL ... scenario reflections

- Early Testing (or concurrent testing). e.g. from scenario ...
  - ECU tested extensively before a real engine prototype available. Allows engineers to identify and fix software bugs early on, boost development, saving time and money.
- Comprehensive Testing
  - Simulated engine can be subjected to range of operating conditions (e.g. high temperature, fault conditions) that would be difficult or dangerous to recreate with a real engine.
- Repeatability, Ease and Accuracy of testing
  - Tests can be repeated precisely, easier to isolate and analyze issues. Also easier to test instead of field testing using actual can / engine.
- Safety:
  - Dangerous scenarios (e.g. engine misfiring, blowing up or such) can be tested without risk to real engine or the testing personnel.
- Cost Savings (could be a lot saved!):
  - HILT testing can be more cost-effective, not repeatedly building and testing prototypes.

**Study comments:** You should know what HITL is about! For EEE3096S we don't really have opportunity to do a prac or project on this. But the concept may likely appear in the exam; about this very important metho is of benefit and you should be able to discuss how it could be applied for a particular embedded system application example.

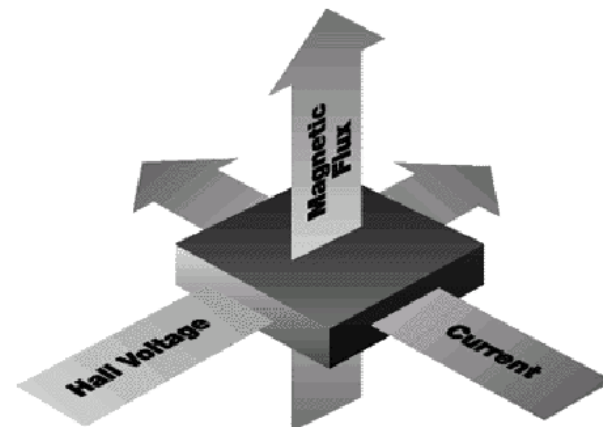
# Drawbacks of HITL

- Use of HITL needs to be properly planned, is not necessarily appropriate for all projects
- It can be costly, take much development time (possibly outweighing potential advantage)
- May have limited use / maintenance difficulty
  - e.g. slight changes to physical engine or vehicle it is in may invalidate the simulation methods, possibly needing massive changes and extraneously much time spent on the HILT instead of the ECU itself.
- These are just some of the main drawbacks



# Examples of common sensors used in HITL

- Rain sensors for wiper control  
("Sensors multiply like rabbits" [ITT automotive])
- Pressure sensors
- Proximity sensors
- Engine control sensors
- Hall effect sensors





# Sampling

Embedded Systems do much sampling, which involves converting detected signals to data (not always using an ADC), that can then be processed, provides information of some sort, from which decisions and actions may be performed.

# Signals and Sensing

- Sensors detect aspects in the environment, possibly signals, movement or characteristics within different media (e.g. vibration, pressure)
- Sensors generate signals that computers can use
- Definition: a signal  $s$  is a mapping from the time domain  $D_T$  to a value domain  $D_V$

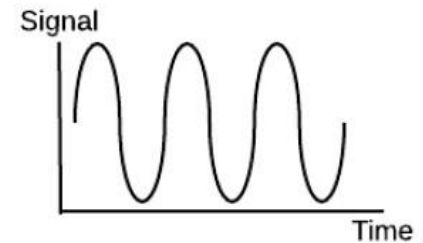
$$s : D_T \rightarrow D_V$$

where

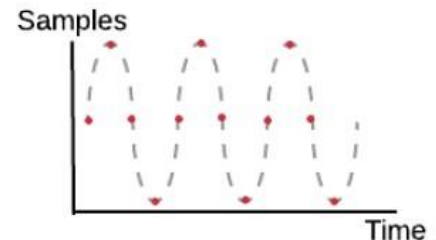
$D_T$ : continuous or discrete time domain

$D_V$ : continuous or discrete value domain

Real sampled  
signal



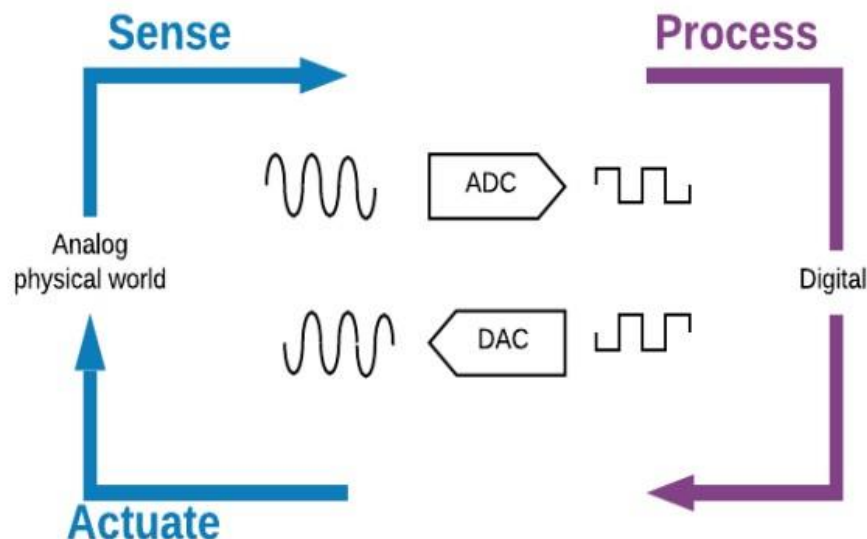
Series of Samples  
providing digital  
(approximated)  
representation of  
signal





# Reasons for Sampling and Digital Signals

- Digital signals are:
  - Less susceptible to noise
  - Easier to process (by computer)
  - Easier to store



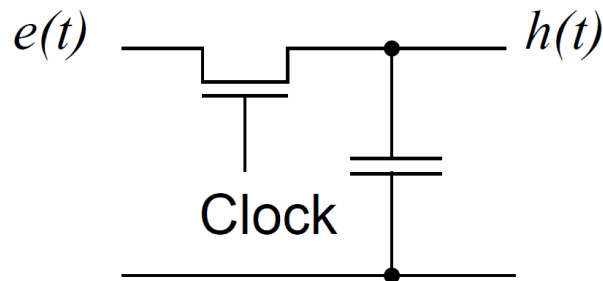
*BUT* a digital signal are discrete and have a finite representation of a signal at a certain time instance. Whereas real signals are continuous. Thus real signals tends to be approximated (sometimes very accurately) by a digital representation.

... and this leads us to the S/H circuit...

# Sample-and-hold circuits (also called 'S/H')

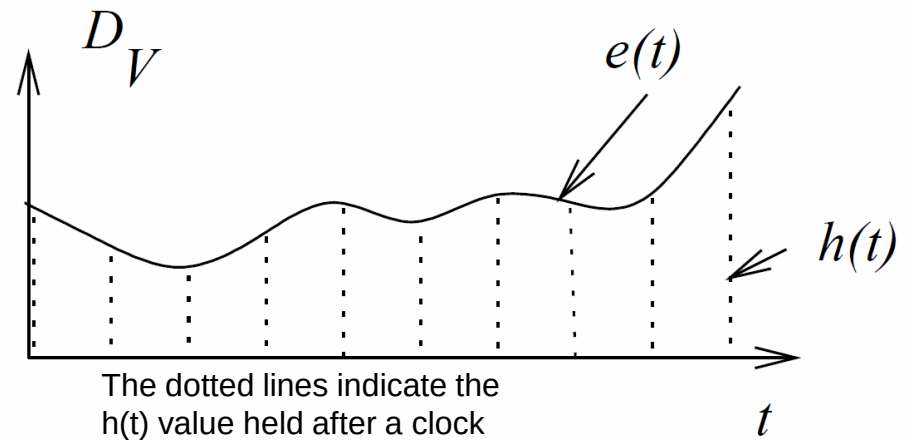
Clocked transistor + capacitor;  
Capacitor stores sequence values

A good S/H is an essential part of an ADC because you ideally want to represent the voltage at a certain instant.



Simple S/H implementation

$e(t)$  is a mapping  $\mathbb{R} \rightarrow \mathbb{R}$



$h(t)$  is a **sequence** of values or a mapping  $\mathbb{Z} \rightarrow \mathbb{R}$

Basically what we are saying here is that you've got a certain countable number of instances that map to a value.

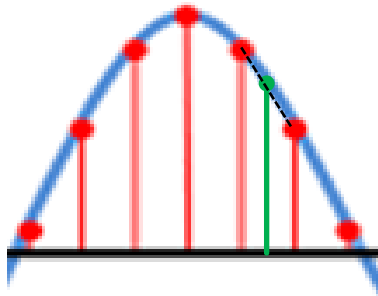
*Where  $\mathbb{Z}$  is of course integers (negative, zero, and positive)*

# **Information Theory & Sampling**

(a brief recap)



## Simon's Guide To The Essence of Sampling (recap)



EEE3096S

# SAMPLING

## Embedded Systems II

# Sampling

Q: Do we lose information due to sampling? ...

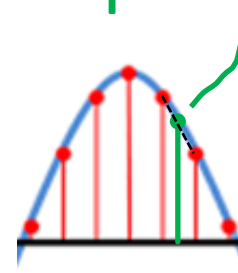
Well, this question might seem to most like asking  
“Is the pope catholic?”

A: In case you are wondering, the answer I’m looking  
for is: **Yes!**

But we may not need all the information that is lost,  
what we sample may be sufficient. For example, high  
logic may be +5V and low logic may be 0V with a +/-  
1V tolerance. So if we got a 5.1V that extra 0.1V is not  
carrying any useful information.

# Sampling & Interpolation

- Sampling and quantization are important:
  - This translates the signal from the analog world to the digital world
- Sampling happens at **particular instances**\*
- When the goal is to determine the value of the signal at some particular point in time, it may be necessary to use **interpolation** between actual samples.



*\* For all intents and purposes we can say 'instance' although the physics implies it is actually an averaged value over a tiny period starting at a particular moment.*

# Shannon's Theorem

- Shannon's Theorem gives an **upper bound** to the capacity of a link, in bits per second (bps), as a function of the available bandwidth and the signal-to-noise ratio of the link.
- The Theorem can be stated as:

$$C = B * \log_2(1 + S/N)$$

where C is the achievable channel capacity,  
B is the bandwidth of the line,  
S is the average signal power and  
N is the average noise power.

Example...



# Example (Shannon's Theorem)

- For a typical telephone line with a signal-to-noise ratio of 30dB and an audio bandwidth of 3kHz, what do we get as the maximum data rate? ...

*Do this quickly on paper or using your calculator...  
how many kbps is needed*

**Answer...**

**Using  $C = B * \log_2(1 + S/N)$**

$$S/N \text{ dB} = 10 \log_{10}(S/N) \rightarrow S/N = 10^3$$

$$C = 3000 * \log_2(1001)$$

$$= 29901 \text{ bps}$$

→ little less than **30 kbps**.



Eish! But we've done that so much before!

It's a plug-and-play exercise...

where C is the achievable channel capacity,  
B is the bandwidth of the line,  
S is the average signal power and  
N is the average noise power.

*Hint S/N is essentially given already but may need to de-log it*

# Limitations of Shannon's Theorem

- The theorem gives **max channel capacity** for an 'archetypal' communication link
- It is a bound on the maximum amount of error-free digital data (i.e. information) that can be transmitted in a specified bandwidth given a certain amount of noise interference

In reality there may be many factors to make it impossible to achieve this 'Shannon optimal' e.g. imperfect sensors, timer drift, etc.

BUT technically you could also get 'super Shannon' rates using compression etc.





Good      Bad  
Nyquist    Nyquist

# The Embedded Engineers Best Friend & Worst Nemesis

# Nyquist's Theorem

- Nyquist's Theorem:
  - “Any signal can be represented by discrete samples if the sampling frequency is at least twice the bandwidth of the signal.”
  - i.e.  $f_{\text{sampling}} > 2 \times f_{\text{signal}}$

where  $f_{\text{sampling}}$  is sampling rate and  $f_{\text{signal}}$  is the maximum frequency of the signal you want to sample.

Further reading: <http://whatis.techtarget.com/definition/Nyquist-Theorem> ,  
[https://en.wikipedia.org/wiki/Nyquist%E2%80%93Shannon\\_sampling\\_theorem](https://en.wikipedia.org/wiki/Nyquist%E2%80%93Shannon_sampling_theorem)

Eish! We've done it too much already!

I know but it's worth remembering ☺





# Nyquist's Theorem

- A *bandlimited* analog signal that has been sampled can be perfectly reconstructed from an infinite sequence of samples if the **sampling rate  $f_s$  exceeds  $2f_{max}$**  samples per second, where  $f_{max}$  is the highest frequency in the original signal.
  - If the analog signal does contain frequency components larger than  $(1/2)f_s$ , then there will be an aliasing error.
  - **Aliasing:** when the digital signal appears to have a different frequency than the original analog signal.

# Difference between: Shannon and Nyquist's theorems

- **Nyquist's Theorem** deals with the concept of sampling rates
  - This is extremely important for embedded engineers, since the sampling needs to be twice as fast as the speed of the signal changes that you want to detect.
  - The general rule of thumb is that your sampling rate must be at least 2x the frequency of the signal you are sampling.
- **Shannon's Theorem** has to do with throughput
  - No cable or signal is perfectly clean. There is a lot of interference from other sources that cause noise on the line. What Shannon's theorem does is predicts the speed of the data that you can push through the cable before noise becomes too much of a factor.  
(Looking at this aspect may be useful in the ECE design project)

Summary: Nyquist is all about sampling. Shannon is all about noise. They are related as speed of data impacts both theorems.

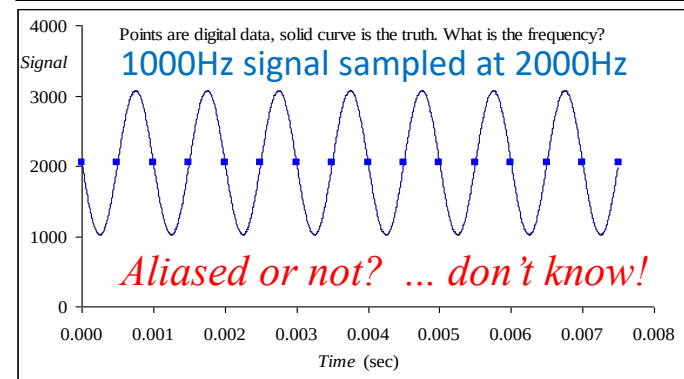
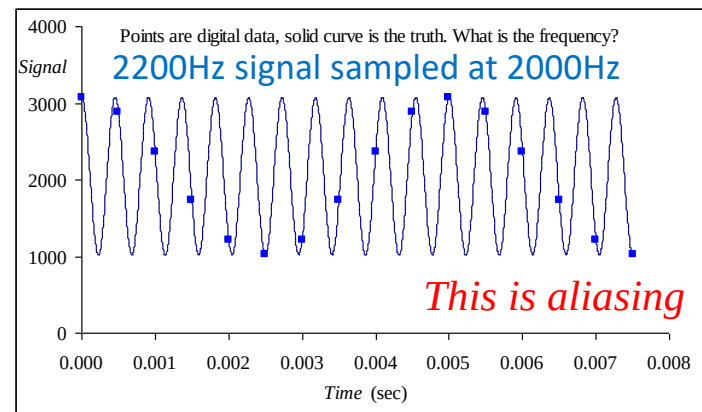
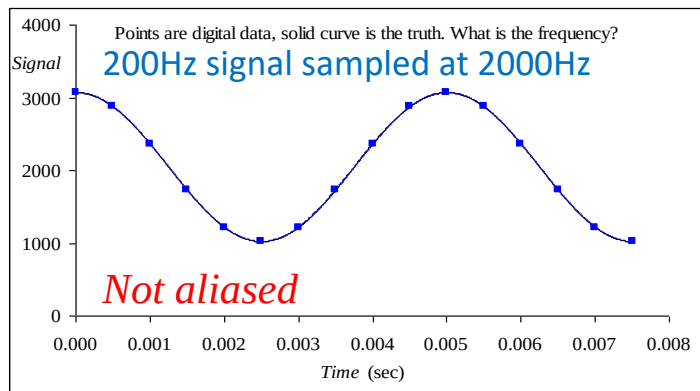
# Shannon & Nyquist Relevant to ADCs

- These formulae are useful tools for planning what ADCs you need and what your system will be capable of in terms of sampling and utilizing channels
  - Shannon will tell you the most you can get out of your channel (excluding 'super Shannon')
  - Nyquist will tell you what frequencies you can expect to get out for a certain ADC  
(assuming the ADC is linear and pretty perfect, we will see later how ADCs may be imperfect which may mean you might not even be able to reliably sample  $F_s/2$  frequencies)

# Sampling and aliasing

- **Aliasing** is an effect that causes different signals to become indistinguishable (or aliases of one another) when sampled.
- It also refers to the distortion or artefact that results when the signal reconstructed from samples is different from the original continuous signal.

Some examples...





# Towards ADCs...

- You probably suspected already... this is all getting your ready to dive into further aspects of ADCs, as essential part of many embedded products.

Next Episode:

## ADCs



Analogue to Digital Convertors (ADCs)

# The Next Episode...

## Lecture L8

ADCs and ADC Metrics :  
Design essentials of ADCs and measuring  
how good your ADC.

